```
Task DeleteAsync(int id);
```

Then add an implementation to the `PropertyService` class.

Listing 8.37 The `Delete` method implementation

```
public async Task DeleteAsync(int id)
{
    var property = new Property { Id = id, Deleted = DateTime.Now };      ❶
    _context.Attach(property).Property(p => p.Deleted).IsModified = true; ❷
    await _context.SaveChangesAsync();                                    ❸
}
```

❶ Create a stub to represent the item to be modified.

❷ Attach the entity to the context, and specify which property should be modified.

❸ Commit the changes.

This method provides another example where stubs are useful. You only want to update the `Deleted` column in the database for this entity. If you pass the entire entity to the `Update` method, all properties are included in the generated SQL `UPDATE` statement. To avoid this, you can use `Attach` to tell the context to start tracking your entity with its state set to `Unchanged`. Then you explicitly set the `Deleted` property as modified. When you set individual properties as modified, only those properties are included in the `UPDATE` statement. You utilize this method by replacing the injected context in the `DeleteModel` with the `IPropertyService`.

Listing 8.38 Modified `DeleteModel` takes a dependency on `IPropertyService`

```
public class DeleteModel : PageModel
{
    private readonly IPropertyService _propertyService;      ❶
```